Two Sum II (Sorted Input) – HashMap vs Two Pointers

In Two Sum II, the array is already sorted. We can use a HashMap, but interviewers usually prefer the two-pointer approach because it uses constant extra space and takes advantage of the sorted order.

HashMap approach
Time: O(n)
Space: O(n)
It stores previously seen values and checks whether the required complement exists.

Two-pointer approach
Time: O(n)
Space: O(1)
Start with one pointer at the beginning and one at the end. Because the array is sorted, if the sum is too small we move the left pointer right, and if the sum is too large we move the right pointer left.

Example
numbers = [2, 7, 11, 15], target = 9
2 + 15 = 17 (too large) → move right
2 + 11 = 13 (too large) → move right
2 + 7 = 9 → answer found

Why return left + 1 and right + 1?
In LeetCode Two Sum II, the problem asks for **1-based indices**, not 0-based indices. Java arrays use 0-based indexing, so when the correct pair is at positions 0 and 1, we must return 1 and 2.

Example
Array: [2, 7, 11, 15]
left = 0, right = 1
Return [left + 1, right + 1] = [1, 2]

Rule of thumb
Unsorted array → HashMap
Sorted array → Two pointers

Key takeaway
HashMap is not wrong for Two Sum II. The two-pointer solution is preferred because it uses the sorted property of the array and reduces extra space from O(n) to O(1).